

verifier et ajouter les commentaire from flask import
jsonify, request

```
ERRORS = {  
    "COURSE_NOT_FOUND": ("Cours introuvable", 404),  
    "INVALID_BODY": ("Le corps de la requête est  
    invalide", 400),  
    "MISSING_FIELDS": ("Des champs obligatoires sont  
    manquants", 400),  
}
```

Afficher plus ▾

Ton code est déjà clair, mais on peut améliorer les commentaires pour qu'ils soient plus précis et homogènes. Voici une version commentée :

```
from flask import jsonify, request  
  
# Dictionnaire des erreurs de l'API.  
# Chaque entrée associe un code d'erreur à :  
# (message retourné au client, code HTTP)  
ERRORS = {  
    "COURSE_NOT_FOUND": ("Cours introuvable", 404),  
    "INVALID_BODY": ("Le corps de la requête est invalide", 400),  
    "MISSING_FIELDS": ("Des champs obligatoires sont manquants", 400),  
}  
  
def get_error(code: str):  
    """  
    Construit une réponse JSON d'erreur à partir d'un code.  
  
    Args:  
        code (str): Code d'erreur défini dans ERRORS.  
    """
```



ChatGPT ▾

✦ Mettre à niveau



▶ Exécuter ↗



- Réponse Flask au format JSON.
- Code de statut HTTP associé.

Si le code n'existe pas dans ERRORS,

```
une erreur interne (500) est renvoyée.
"""

message, http_status = ERRORS.get(
    code,
    ("Erreur interne", 500)
)

return jsonify({
    "error": message,
    "code": code
}), http_status


def get_json_body():
    """
    Récupère et valide le corps JSON de la requête.

    Returns:
        dict | None:
            - Le corps JSON sous forme de dictionnaire.
            - None si aucun JSON n'est fourni, si le JSON est invalide
              ou si le contenu n'est pas un objet JSON.
    """
    # Lecture silencieuse du JSON.
    # En cas d'erreur de parsing, Flask retourne None au lieu de lever une exception.
    body = request.get_json(silent=True)

    # On attend uniquement un objet JSON (dictionnaire).
    if not isinstance(body, dict):
        return None

    return body


def validate_course_body(body: dict):
    """
    Vérifie que le corps de la requête est valide pour un cours.

    Conditions vérifiées :
        - Le body existe.
        - Tous les champs obligatoires sont présents.
        - Les types des champs sont corrects.
        - Les valeurs numériques sont positives ou nulles.

    Args:
        body (dict): Corps JSON de la requête.

    Returns:
        str | None:
            - Un code d'erreur ("INVALID_BODY" ou "MISSING_FIELDS")
              si une validation échoue.
            - None si le body est valide.
    """
```

```
.....

# Vérifie que le body est présent.
if body is None:
    return "INVALID_BODY"

# Liste des champs obligatoires.
required_fields = ["name", "coach_name", "seats", "price"]

# Vérifie que tous les champs sont présents.
for field in required_fields:
    if field not in body:
        return "MISSING_FIELDS"

# Validation des types.
if not isinstance(body["name"], str):
    return "INVALID_BODY"

if not isinstance(body["coach_name"], str):
    return "INVALID_BODY"

if not isinstance(body["seats"], int):
    return "INVALID_BODY"

# Le prix peut être un entier ou un flottant.
if not isinstance(body["price"], (int, float)):
    return "INVALID_BODY"

# Les valeurs numériques doivent être positives ou nulles.
if body["seats"] < 0:
    return "INVALID_BODY"

if body["price"] < 0:
    return "INVALID_BODY"

# Toutes les validations sont passées.
return None
```

Quelques améliorations apportées

- Ajout de **docstrings complètes** (Args , Returns).
- Explication du rôle du dictionnaire ERRORS .
- Commentaires avant chaque étape importante (lecture du JSON, validation des champs, validation des types, validation des valeurs).
- Uniformisation du style de commentaires pour respecter les bonnes pratiques Python.

